Tree Mapping App: Game Plan

Oct 25, 2026 · @D

Build a web-first app on open map tech (MapLibre, OpenStreetMap-based tiles, Postgres/PostGIS), seed it with open city tree registers, then add a native mobile app once the capture flow works.

Short answers to your questions

How do we get map functionality? You embed a map; you don't build one. Use MapLibre GL, an open-source map renderer, with free vector tiles made from OpenStreetMap, and draw your trees as a layer on top. Mein Baum itself uses OSM as its base map, so no Google Maps bill or API key is needed.

Is there scope for both a web app and a mobile app? Yes, and one backend can serve both. Build the web app first as an installable PWA (camera and GPS work in the phone browser), then add a native app when you need offline use, background location or push notifications.

Assumption: the plan starts with one pilot city, because open tree registers differ by city. If you tell me which region you are targeting, the data section can be narrowed.

What Mein Baum does, and where you can differ

Mein Baum is a citizen-science app launched in April 2025 by BUND Naturschutz with TUM and HSWT; volunteers record city trees by GPS, photo and measurements. It runs on the SPOTTERON citizen science platform, so its owners did not write a backend from scratch. You can do the same or build your own.

Features worth copying:

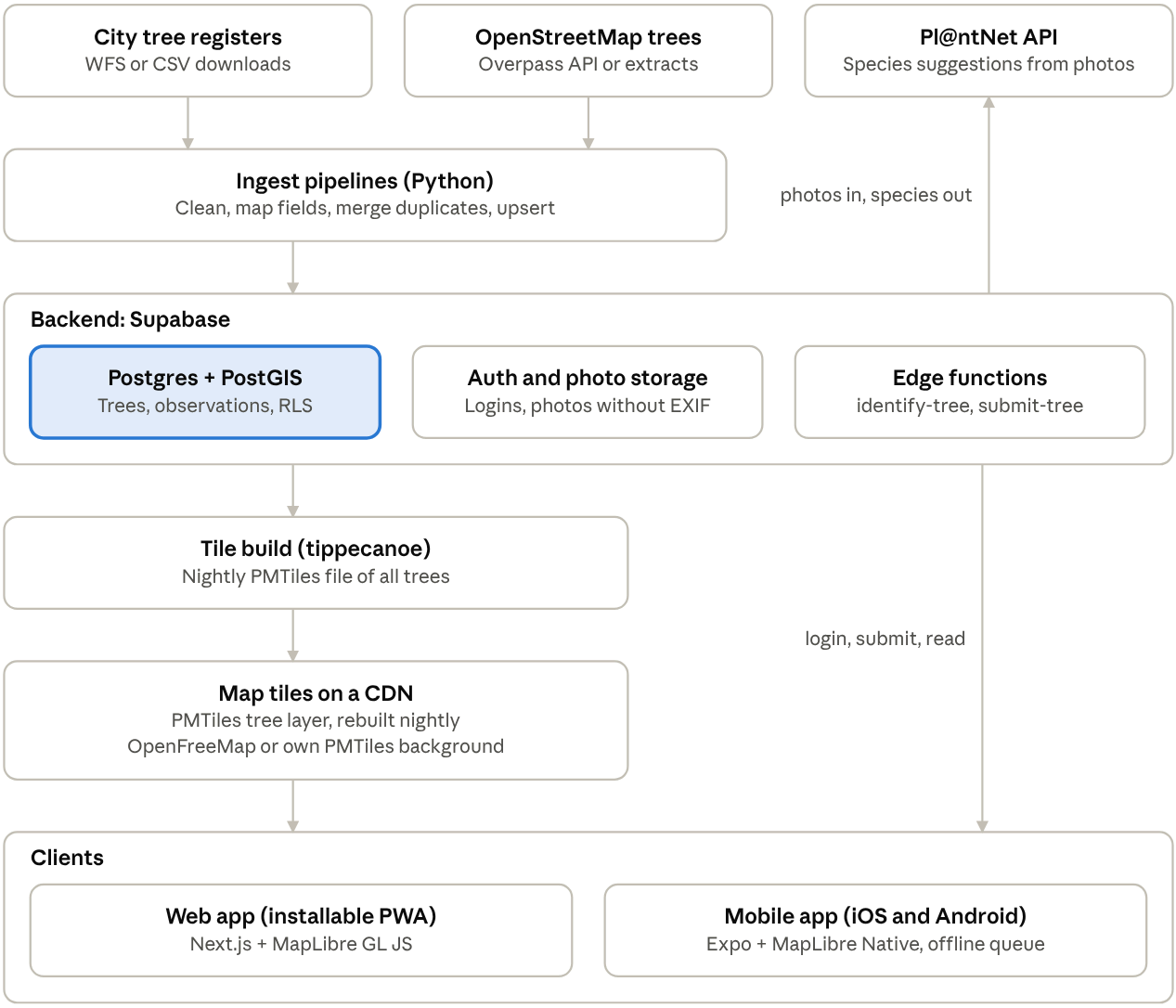
- Record a tree's location, species, surroundings, root zone, trunk and crown, per the Regensburg launch report.
- Photo upload so experts can check each entry.
- A community area with actions such as watering trees in dry spells.
- A map showing which trees suffer from drought.

Where a new app can add value:

- Mein Baum entries did not show up in OSM, [according to an OSM forum thread](). Publishing your data openly, or syncing with OSM, is a clear point of difference.
- Forum members note that volunteers cannot match a city's full register or keep it current. Start from the city register and let citizens correct and extend it.

Architecture

One backend and one tile pipeline feed both the web and the mobile app



architecture · sources, backend, tiles, two clients

Read top to bottom: open data and OSM trees are cleaned into Postgres, tiles are built from it nightly, and both apps read the same tiles and call the same API.

Map functionality: the stack

Use MapLibre to draw the map, free OSM-based vector tiles for the background, and tiles generated from your own database for the tree layer. Every piece is open source and needs no map-vendor account.

Layer	Choice	Notes
Map renderer	MapLibre GL JS on web; MapLibre Native on iOS/Android	Wrappers exist for Flutter and React Native. It renders vector tiles on the GPU.
Background map	OpenFreeMap public instance, or your own PMTiles file	OpenFreeMap needs no key or registration and is funded by donations, so keep a self-hosted fallback.
Self-hosted tiles	PMTiles on any file or object storage	One file, read by HTTP range requests, no tile server to run. The same file can be bundled for offline use.
Tree layer, one city	Vector tiles from PostGIS (Martin or pg_tileserv), or a PMTiles file built with tippecanoe	Berlin alone lists over 900,000 active trees, too many for plain GeoJSON.
Tree layer, small area	GeoJSON with clustering	Fine for a few thousand points; use for the first prototype.
Place search	Nominatim for light use, or a hosted geocoder	Respect the public Nominatim usage policy.

Three rules avoid common problems:

- Do not point a production app at the volunteer-run tile servers on openstreetmap.org; use a tile host above.
- Show the attribution OSM and OpenMapTiles require ("© OpenStreetMap contributors").
- Phone GPS is off by several metres between buildings, so let people drag the pin and show nearby existing trees to prevent duplicates.

Web app or mobile app: build the web app first

A web app lets you test the product with real users in weeks, because there is no app-store review and one deploy updates everyone. Add the native app once users ask for offline mapping or push reminders.

Need	Web app (PWA)	Native app
Getting users	Open a link, no install; shareable URL per tree	App-store install and review
Camera and GPS	Works in the phone browser	Works, with finer control
Offline map and queued edits	Possible but limited storage	Reliable; best for field mapping
Push reminders (watering)	Limited, varies by phone	Full support
Search visibility	Tree and city pages can be indexed	None
Effort on top of the other	Baseline	Extra build, store setup and offline sync work; a shared API client and types reduce it

Recommended stack (one TypeScript monorepo): Next.js for the web app, Expo (React Native) with MapLibre for mobile, and Supabase for Postgres with PostGIS, login, photo storage and row-level security. AI coding agents handle this combination well, and web and mobile share the API client and types.

Alternative: Flutter with the maplibre_gl plugin, which targets Android, iOS and web from one codebase. It is simpler to staff, but the web build is weaker for search visibility.

Also consider: a white-label citizen-science platform such as SPOTTERON, which Mein Baum uses, if you want to launch with no code at all.

Where to get the data

Seed the map from a city's open tree register, add OpenStreetMap trees, and let citizens fill the gaps. Check each licence before you load a register, because terms differ by city.

Source	What you get	Access	Terms
Berlin Baumkataster	Over 900,000 active street and park trees, not all with coordinates yet; no private trees, cemeteries or forests	Geoportal Berlin web services (WFS) and downloads	Check the dataset page
Hamburg Straßenbaumkataster	About 227,000 street trees with species and planting year	Transparenzportal Hamburg	Check the dataset page
Vienna Baumkataster	Tree locations with species	data.gv.at download	CC-BY 4.0
Bern Baumkataster	City tree register	WMS and WFS services	Commercial use allowed, credit required
NYC Street Tree Census 2015	666,134 street trees with species, diameter, health	NYC Open Data download and API	Licence field says unspecified; check
OpenStreetMap trees	Points tagged natural=tree, often with genus, species, circumference, height	Overpass API, regional extracts	ODbL; credit and share-alike apply
Pl@ntNet API	Species suggestions from photos	REST API with a free key	Free to 500 identifications a day; commercial use above that is €1,000 a year (excl. tax) for 200,000 requests; credit line requested

Other German cities that published their registers as of 2017 include Frankfurt, Leipzig, Köln and Heilbronn; check their portals for the current state. For other countries, search national open-data portals for "street tree inventory" or the local word for it.

Two licensing traps:

- A city register under CC-BY is not automatically allowed into OSM. OSM mappers question this for the Vienna data, so ask the city before you copy trees into OSM.
- Store a `source` and `licence` on every tree record, so you can remove or credit a dataset later.

Not checked in this research: GBIF and iNaturalist for species names and images. Both are standard, but confirm their API terms first.

Data model

Keep one row per tree and many dated observations per tree, so health and size can be tracked over time. Name fields after the OSM tree tags so export to OSM stays easy.

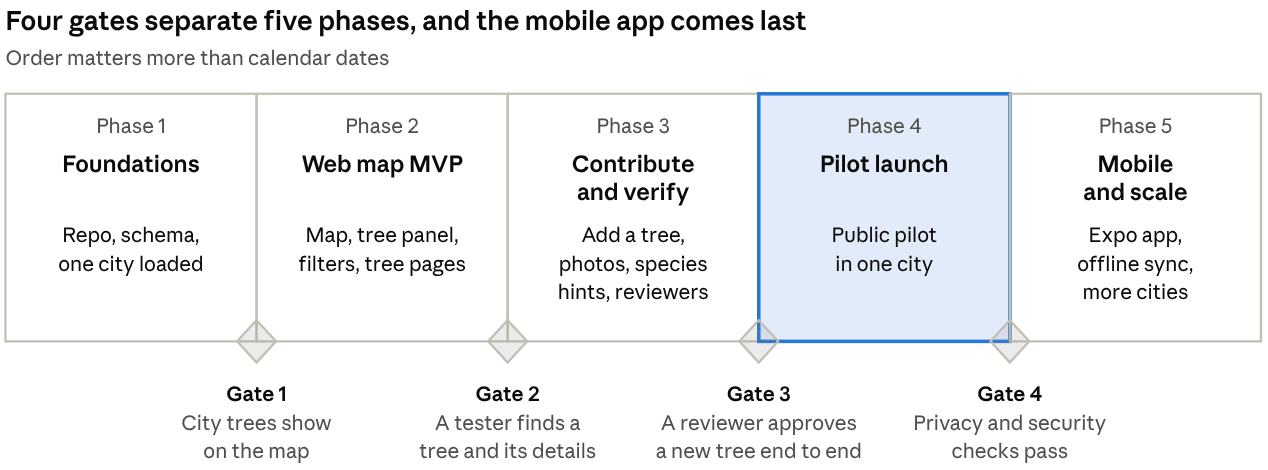
Field	Type	Note
id	uuid	Primary key
geom	Point, EPSG:4326	PostGIS geography, spatial index
genus, species	text	Latin names, e.g. Quercus robur; may be empty until identified
leaf_cycle	enum	deciduous or evergreen
circumference_m	number	Trunk at 1.3 m height, as in OSM
height_m, crown_diameter_m	number	Measured or estimated; add an `estimated` flag
planted_year	integer	From city registers
source	enum	city_register, osm, citizen
source_id, licence	text	Keep for credit and removal
status	enum	unverified, verified, disputed, removed

Field	Type	Note
created_by, created_at, updated_at	user, timestamps	Audit trail

Related tables:

- **observations:** tree id, date, health (good, fair, poor, dead), surroundings and root-zone notes, measurements, author.
- **photos:** tree or observation id, storage path, author, taken_at.
- **verifications:** observation id, reviewer, decision, comment.
- **care_events:** tree id, user, type (watering, pruning), date; this powers the community watering feature.

Roadmap



roadmap · 5 phases, 4 gates

Do not start the next phase until the gate before it is met. Prompts 1 to 3 belong to phase 1, 4 to 6 to phase 2, 7 to 9 to phase 3, 11 and 12 to phase 4, and 10 to phase 5.

Prompts for coding agents

Give the agent the shared context first (save it as AGENTS.md or CLAUDE.md in the repo root), then one module prompt at a time, in order. Replace anything in [square brackets]. Each prompt ends with acceptance criteria, so you can check the result before moving on.

0. Shared project context

```
# Project: TreeMap (working name)
A citizen-science web and mobile app to map and monitor city trees. It is
seeded from open city tree registers and OpenStreetMap, then improved by
citizens with photos and measurements, and checked by reviewers.

Stack: TypeScript monorepo (pnpm + Turborepo).
- apps/web: Next.js (App Router)
- apps/mobile: Expo (React Native)
- packages/types, packages/api-client
- services/ingest: Python 3.12 data pipelines
- supabase/: migrations and edge functions (Postgres + PostGIS)
Map: MapLibre GL (maplibre-gl on web, @maplibre/maplibre-react-native on
mobile). No Google Maps, no Mapbox.

Rules:
- Every tree keeps provenance: source, source_id, licence, status. Never drop
them.
- Name fields after OSM tree tags: genus, species, circumference_m (trunk at
1.3 m), height_m, crown_diameter_m, leaf_cycle.
- v1 covers public-land trees only. Strip EXIF and GPS from uploaded photos.
- Strict TypeScript. Validate every API input with Zod. Row Level Security on
every table.
- Write tests with the code (Vitest, pytest). Keep commits small.
- Record each architecture decision in docs/decisions.md.
- Ask before adding a dependency whose licence is not MIT, Apache-2.0 or BSD.
```

1. Repo scaffold and CI

```
Using the project context, create the monorepo skeleton.
1. pnpm workspaces + Turborepo with the folders listed in the context.
2. apps/web: Next.js + TypeScript + Tailwind, a home page that renders an
empty MapLibre map centred on [CITY, lat, lon].
3. apps/mobile: Expo + TypeScript with a placeholder screen. Do not add map
code yet.
4. Shared ESLint, Prettier and tsconfig. A root README with setup steps.
5. GitHub Actions: lint, typecheck and test on every pull request.
6. .env.example listing every variable, with no secrets committed.
Acceptance: a fresh clone runs `pnpm i && pnpm dev` and shows the map on
localhost; CI passes on a test pull request.
```

2. Database schema and security

```
Create Supabase SQL migrations under supabase/migrations.
Tables: trees, observations, photos, verifications, care_events, profiles.
- trees.geom is geography(Point,4326) with a GIST index. Add source (enum:
city_register, osm, citizen), source_id, licence, status (unverified,
verified, disputed, removed), genus, species, leaf_cycle, circumference_m,
height_m, crown_diameter_m, planted_year.
- observations: health enum (good, fair, poor, dead), surroundings text,
notes, author, observed_at.
- Unique index on (source, source_id) to make imports repeatable.
Row Level Security: everyone reads trees and verified photos; signed-in users
insert observations and photos; only the author edits their own rows;
reviewers (a role in profiles) write verifications; only the service role
writes imports.
Add a SQL function trees_near(lon, lat, radius_m) that returns existing
trees, used to prevent duplicates.
Acceptance: migrations apply cleanly on a blank database; pgTAP or SQL tests
prove each RLS rule; generate TypeScript types into packages/types.
```

3. Import a city tree register

```
In services/ingest, write a repeatable Python importer for the tree register
of [CITY] from [DATASET URL or WFS endpoint].
1. Download the data (WFS GetFeature as GeoJSON, or the CSV/Shapefile) into a
local cache. Never commit raw data.
2. Reproject to EPSG:4326 and drop rows without a valid point.
3. Map the city's columns to our schema in a mapping file (YAML), not in
code. Split a combined Latin name into genus and species; convert trunk
circumference or diameter to metres; keep the original value in a
raw_attributes JSON column.
4. Upsert on (source, source_id) so a second run updates rather than
duplicates. Set source=city_register and licence=[LICENCE TEXT].
5. Print a report: rows read, loaded, skipped (with reasons), species with no
match.
Acceptance: running twice gives the same row count; a pytest suite covers the
mapping on a 50-row sample file; the report is saved to out/report.md.
```

4. Add OpenStreetMap trees and merge duplicates

```
In services/ingest, add an importer for natural=tree nodes inside the
bounding box of [CITY].
1. Query the Overpass API (or read a regional extract) and cache the result.
Be polite: one query per run, a descriptive User-Agent, retry with back-off.
2. Map tags to our schema: genus, species, circumference (metres), height,
diameter_crown, leaf_cycle. Reject impossible values, for example a
circumference above 43 m or a height above 116 m.
3. Source=osm, source_id=the OSM node id, licence=ODbL with credit text "©
OpenStreetMap contributors".
4. Merge step: if an OSM tree lies within [3] m of a city_register tree, link
them in a tree_links table instead of inserting a second tree. Fill empty
fields on the register tree from OSM and never overwrite register values.
5. Report counts of inserted, linked and rejected trees.
Acceptance: tests cover the plausibility rules and the 3 m matching with
fixture data; re-running is idempotent.
```

5. Tile pipeline for the tree layer

```
Create a pipeline that publishes the tree layer as vector tiles.
Option A (preferred): export trees to GeoJSON-seq, build a PMTiles file with
tippecanoe (zoom 10-18, keep genus, species, status, height_m; cluster points
below zoom 14), upload it to object storage with public read and CORS for
range requests.
Option B: serve tiles from PostGIS with Martin using a function source.
Implement A in a script `pnpm tiles:build`, plus a GitHub Action that
rebuilds nightly. Write docs/tiles.md describing both options and when to
switch to B (when edits must show up instantly).
Acceptance: the PMTiles file loads in the web map from a public URL; the
build log shows tile count and file size; a city of 100,000+ trees stays
under [50] MB.
```

6. Web map app

```
In apps/web, build the main map screen.
- MapLibre GL with the background style from [OpenFreeMap liberty style URL or our PMTiles style]. Attribution control visible.
- Tree layer from the PMTiles source: circles coloured by leaf_cycle, sized by zoom, clustered at low zoom.
- Click a tree to open a side panel (bottom sheet on phones) showing species, size, planting year, health, photos, source and licence credit, and a link to its own page /tree/[id].
- Filters: species/genus search, health, verified only. Persist the map position and filters in the URL.
- Locate-me button using the browser Geolocation API.
- Responsive from 360 px wide; keyboard accessible; labels for screen readers.
- A server-rendered /tree/[id] page with title, description and Open Graph tags.
Acceptance: Lighthouse performance and accessibility scores of at least 90 on the map page on a mid-range phone profile; Playwright test opens the map, clicks a tree and sees the panel.
```

7. Add-a-tree and update flow

```
Implement the contribution flow shared by web and mobile (put logic in packages/api-client, UI in each app).
1. Step 1, place: use the device location, let the user drag the pin, and show trees within 10 m using trees_near(). If one is within [3] m, ask "Is this the same tree?" and offer to update it instead of creating a new one.
2. Step 2, photos: 1-4 photos (whole tree, bark, leaf, crown). Resize on the device, strip EXIF, upload to storage under the user's folder.
3. Step 3, details: species (see the species suggestion prompt), trunk circumference, height estimate, health, surroundings, notes. All optional except location and one photo.
4. Submit creates a tree (status=unverified) plus an observation, in one transaction through an RPC.
5. Rate limit: 30 submissions per user per day.
Acceptance: Zod schemas shared by both apps; unit tests for the duplicate check; a Playwright test completes the flow with a fixture image.
```

8. Species suggestions from a photo

```
Add a Supabase edge function `identify-tree` that proxies the Pl@ntNet
identify API.
- The API key stays in a server secret; never ship it to the client.
- Input: 1-4 image paths from our storage and an optional organ hint (leaf,
bark, habit). Output: top 5 suggestions with Latin name, common name, score.
- Limits: the free tier is 500 identifications a day, with a cap on
simultaneous requests (read the current terms; they list 20). Use one server-
side account only, because the terms forbid several free accounts. Add a per-
user limit of [10] a day, a global daily counter that returns a friendly "try
later" when exhausted, and a queue so we never exceed the parallel-call cap.
- Cache results by image hash for 30 days.
- Show the credit line required by the Pl@ntNet terms wherever suggestions
appear.
- If the top score is below [0.3], tell the user the suggestion is uncertain
and offer a manual pick from a list of the 200 most common species in [CITY].
- The suggestion is only a suggestion: store the user's choice plus the
suggested alternatives, so reviewers see both.
Acceptance: tests with a mocked Pl@ntNet server cover rate limits, cache hits
and low-confidence handling.
```

9. Review and verification tools

```
Build a reviewer area in apps/web at /review, visible only to users with the
reviewer role.
- Queue of unverified observations, oldest first, with photos, map position,
suggested species and the contributor's history.
- Actions: approve, correct species, mark disputed, reject with a reason.
Each writes a row to verifications and updates the tree status.
- Trust rule: after [5] approved observations, a contributor's new
observations are marked verified automatically but sampled at [10]% for
review.
- Keyboard shortcuts for approve and next.
- A simple report: items reviewed per week, approval rate, median time to
review.
Acceptance: RLS tests show non-reviewers cannot read the queue or write
verifications; Playwright test approves an item and the tree turns verified
on the public map.
```

10. Mobile app

```
In apps/mobile (Expo), build the field-mapping app on top of packages/api-client and packages/types.
- Map screen using @maplibre/maplibre-react-native with the same style and PMTiles tree layer as the web app. Show the user's location and heading.
- The add-a-tree flow from prompt 7 using the camera and GPS, with a draggable pin.
- Offline: download the map for the visible area (as PMTiles or offline region), and queue new observations and photos locally; upload when the connection returns, with retry and a visible "waiting to upload" count.
- Sign-in with Supabase Auth (email link and Apple/Google sign-in).
- Build profiles for development and production with EAS; no secrets in the app bundle.
Acceptance: a manual test script in docs/mobile-test.md; airplane-mode test adds 3 trees and all 3 sync on reconnect with no duplicates.
```

11. Community: watering and care

```
Add care features on top of care_events.
- "I watered this tree" button with date and litres (optional); show "last watered" and a heat/dry-spell indicator on the tree panel.
- A map filter "needs water": trees with no care event in [14] days during a dry spell. Get the weather signal from [Open-Meteo or the national weather service API] with a daily cached job.
- Optional push or email reminders to people who "adopted" a tree, with one-click unsubscribe.
- A basic leaderboard by neighbourhood, opt-in only.
Acceptance: tests for the needs-water query; reminders respect unsubscribe; no personal location of the user is exposed, only the tree's.
```

12. Privacy, security, quality and launch

```
Prepare the app for a public pilot.
1. Privacy: write a plain-language privacy policy and an in-app consent
screen; list personal data (email, photos, contributions); add account export
and account deletion that keeps tree data but removes the author link.
2. Photos: server-side check that EXIF is stripped; reject files above [8] MB
or non-image types; add a report button for inappropriate photos.
3. Security: review RLS with a test that tries forbidden reads and writes as
each role; add rate limits; dependency audit in CI.
4. Accessibility: WCAG 2.2 AA check of the main flows.
5. Observability: error tracking, uptime check, and a dashboard of daily
contributions.
6. Deploy: web on [Vercel], database on Supabase, tiles on object storage
with CDN; write docs/runbook.md for backups and restoring the database.
Acceptance: a launch checklist in docs/launch.md with every item ticked or
explained.
```

Risks to plan for

- **Licences differ by dataset.** City registers use CC-BY or similar; OSM data is ODbL with share-alike. Record the licence per tree and ask each city in writing if the terms are unclear.
- **Crowd data is noisy.** Species from photos can be wrong, and GPS can be several metres off. Keep an "unverified" state, show uncertainty, and recruit local reviewers early.
- **Species identification has limits.** The free PlntNet tier stops at 500 identifications a day. Commercial use beyond that is paid: the terms list €1,000 a year (excl. tax) for 200,000 requests, then €5 per extra 1,000. Non-profit educational or scientific use can be free on request.
- **Privacy.** Photos can show people and number plates, and contribution history reveals where a user walks. Strip EXIF, cover public land only, add a report button, and allow account deletion.
- **Free map hosting can change.** OpenFreeMap is donation-funded, so keep a self-hosted PMTiles background map ready.
- **Reviewer time is the real bottleneck.** Partner with a local nature group; Mein Baum's own partners are BUND Naturschutz and two universities.

Sources

The first three pages were opened in full. The rest were read as search extracts only, so confirm figures and licences on the source page before relying on them.

- Pl@ntNet API terms of use: free tier, pricing, credit line
- Baumbestand Berlin, Geoportal documentation: tree count and coverage
- Hamburg BUKEA: 25 years of the digital tree register: street tree count and fields
- TUM press release on Mein Baum
- Mein Baum on the App Store
- Regensburger Nachrichten on the Mein Baum launch
- OSM community thread on Mein Baum
- OSM community thread on the Vienna tree register
- Bern Baumkataster on opendata.swiss
- NYC 2015 Street Tree Census on Data.gov
- OSM wiki: Tag natural=tree
- Osmose issue on implausible tree values
- OpenFreeMap on GitHub and Simon Willison's note on it
- MapLibre Flutter: PMTiles and Stadia Maps: native map SDKs
- Karlsruhe council motion listing cities with open tree data (2017)